

Multi-RCM Kinematic Control for a ROSA-like Surgical Robot

A Unity Simulation of Task-Space Remote-Center-of-Motion Control
with Target-Centred and Entry-Centred Operating Modes

Samuele Civale – 1938135
Alexandru Vivian Pita – 1948533
Francesca Forghieri – 2217594

July 2026

Abstract

This report documents a Unity-based simulation of a six-degree-of-freedom, ROSA-like manipulator regulating a needle-like tool under a *Remote Center of Motion* (RCM) constraint. The controller is a from-scratch, purely kinematic damped-least-squares (DLS) scheme with a null-space secondary objective and an explicit penetration coordinate λ along the tool shaft, in the spirit of Aghakhani et al. (ICRA 2013) [1]. Four modes are implemented: an entry-centred RCM with a fixed tip target, a target-centred RCM with a conical entry-side motion, a three-phase safe-insertion sequence, and an entry-centred RCM with a conical tip motion around a deep target. All quantitative results are computed from one selected continuous log (`project4_multircm_20260714_193548_422.csv`, 2696 samples, 157.3s) that visits all four modes. Settled entry-RCM error stays sub-millimetre in every regulated task (max 0.151 mm in safe insertion, 0.834 mm in Task 4), and the final insertion tip-to-target error is 0.467 mm, both comfortably inside the clinically motivated 2 mm/3 mm thresholds. Conical-reference tracking is looser (Task 2: mean 4.96 mm on an 8.79 mm commanded radius; Task 4: mean 3.19 mm on a 13.8 mm radius), a direct consequence of pure-proportional feedback with no feedforward term. The implementation is compared aspect-by-aspect with the ICRA 2013 reference formulation, and functionality present in code but inactive in this run (skull-avoidance null-space term, Task-2 joint-4 active-set guard) is identified explicitly.

1 Introduction and Motivation

Stereotactic and minimally invasive neurosurgical procedures – biopsy, electrode placement, needle insertion – require a rigid tool to reach a deep target through a single, pre-planned entry point. Once fixed, any lateral motion of the tool shaft at that opening risks tissue damage along the surgical corridor. The *Remote Center of Motion* (RCM) formalizes “pass through this fixed point while pivoting freely otherwise”: a virtual pivot, external to the robot’s own joints, through which the tool axis must pass while the tip performs a clinically meaningful motion elsewhere. Mechanical RCM devices enforce this passively; a *software*/control-based RCM instead adds a velocity-level task to a general-purpose serial arm, trading mechanical simplicity for a dependence on accurate entry-point registration.

This project implements and evaluates an RCM-constrained kinematic controller in Unity for a procedurally generated, ROSA-like 6-DOF manipulator, built from scratch in C# (no physics engine, no external IK library). It follows the idea introduced by Aghakhani et al. at ICRA 2013 [1] of treating the RCM condition as an explicit penetration coordinate λ solved together with the joint velocities. Four operating modes are implemented, corresponding to two placements of the effective RCM (entry point or target) and a three-phase safe-insertion sequence transitioning between them. The report (i) reconstructs the kinematic model and control logic directly from the C# source, (ii) analyses the tracking and safety behaviour from one selected experimental log, and (iii) compares the approach, aspect by aspect, with the ICRA 2013 reference.

2 Related Work

RCM realizations range from purely mechanical linkages to software-defined constraints solved as part of the inverse-kinematics problem; within the control-based family, RCM satisfaction has been obtained by extending the task Jacobian with an explicit RCM block, by real-time constrained optimization, or by formulating the RCM condition as an equality constraint on a point sliding along the tool shaft. This project follows the last family.

The reference paper is N. Aghakhani, M. Geravand, N. Shahriari, M. Vendittelli, G. Oriolo, “*Task Control with Remote Center of Motion Constraint for Minimally Invasive Robotic Surgery*,” ICRA 2013, pp. 5807–5812 [1]. It expresses the RCM point p_{RCM} as a point sliding, with coordinate $\lambda \in [0, 1]$, along the segment between two consecutive DH frame origins of the link passing through the trocar. Differentiating yields an RCM Jacobian $J_{\text{RCM}}(q, \lambda)$ mapping $(\dot{q}, \dot{\lambda})$ to $\dot{p}_{\text{RCM}}$; imposing $\dot{p}_{\text{RCM}} = 0$ enforces the trocar condition as part of an *extended task* $t_{\text{EXT}} = (t, p_{\text{trocar}} - p_{\text{RCM}})$, solved by $(\dot{q}, \dot{\lambda})^\top = J^\# \text{diag}(K_t, K_{\text{RCM}}) e_t + (I - J^\# J) w$, with $J^\#$ the pseudoinverse of the extended Jacobian and w a null-space vector regulating λ toward a fixed nominal λ_0 . The paper validates this on a simulated visual-servoing task (KUKA KR5, Webots) and a 9-DOF constrained-motion example (DLR MIRO); both are simulation studies. Section 8 compares this formulation systematically with the present implementation.

3 System and Robot Model

The project is a Unity scene built procedurally at runtime by `Project4SceneBuilder` (`Start()`): entry/target markers, an optional skull sphere, the robot and its controller, ground, lighting, and a keyboard-only free-fly camera. Every visual element is generated by the controller itself, so the DH table, gains, and task logic can be changed purely in C# without touching the `.unity` file. In the scene used for the analysed log, `createSkull` is enabled (radius 0.118 m); section 9 clarifies what this sphere does and does not guarantee.

The manipulator is a procedurally generated 6-revolute-joint (6R) arm described by standard DH parameters (table 1), documented in the source as a *plausible ROSA-like* arm, not the manufacturer’s real parameters. The tool is a rigid, non-actuated extension of length $L = 0.283$ m along the wrist z -axis (physical radius 1.2 mm), adding the penetration coordinate λ as a seventh controlled state, $x = (q, \lambda) \in \mathbb{R}^7$. The initial posture is $q_0 = (0, 5.24, -81.16, 0, 59.36, 0)^\circ$.

Table 1: Default DH parameters (`SetDefaultDH()`). $\theta_{\text{offset}} = 0$ for every link.

Link i	a_i [m]	α_i [deg]	d_i [m]	$q_{i,\min}$ [deg]	$q_{i,\max}$ [deg]
1	0.045	−90	0.42	−170	170
2	0.420	0	0.00	−120	120
3	0.055	−90	0.00	−150	150
4	0.000	90	0.36	−170	170
5	0.000	−90	0.00	−120	120
6	0.000	0	0.12	−360	360

Software. Four project-authored C# scripts implement the simulation: `ROSDoubleRCMController.cs` (1656 lines – kinematic model, control law, task/phase state machine, integration, visualization, CSV logging; the core deliverable), `Project4SceneBuilder.cs` (201 lines – scene bootstrap), `FreeFlyCameraKeyboard.cs` (139 lines – viewing aid only), and `RCMDiagnosticPlayerBuilder.cs` (28 lines – editor-only headless-build utility). Offline analysis uses a Python script, `analyze_rcm_logs_clean.py` (698 lines, unmodified), invoked by a thin report-only wrapper on exactly one CSV file. Every 0.05 s (20 Hz) the controller logs tip/RCM/reference positions, all live error metrics, joint angles/velocities, $\lambda, \dot{\lambda}$, task/mode/phase labels, frame time/FPS, and pass/fail flags.

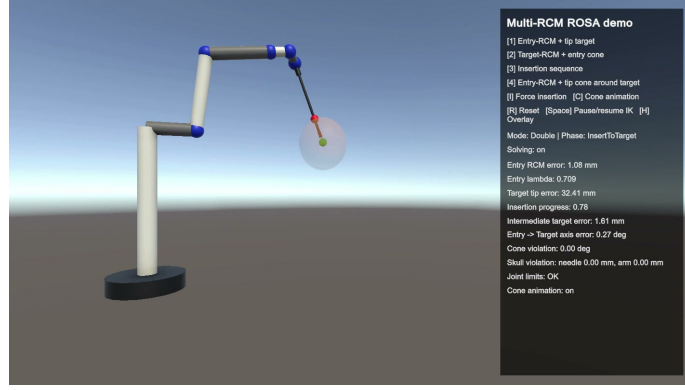


Figure 1: Unity runtime overview: the 6R arm, needle shaft with tip/target markers, translucent skull sphere, and debug overlay. Illustrative screenshot; not tied to the numerical results of section 7.

4 Kinematic Control Framework

Forward kinematics and Jacobians. Given $q \in \mathbb{R}^6$, forward kinematics is the standard product of DH transforms, $T_i(q) = T_{i-1}(q)A_i(q_i)$, giving the tool base point $p_{\text{base}}(q)$ and axis $\hat{z}(q)$; any point at parameter $s \in [0, 1]$ along the tool is $p_{\text{tool}}(q, s) = p_{\text{base}}(q) + sL\hat{z}(q)$, with the tip at $s = 1$. Every task-point Jacobian $J_i(q, \lambda) \in \mathbb{R}^{3 \times 7}$ – tip, RCM point, cone points, insertion-axis guide point – is computed by *central finite differences* (perturbation 10^{-3} on each of the 7 extended-state components), not analytically, even where a closed form exists (e.g. $\partial p_{\text{RCM}}/\partial \lambda = L\hat{z}(q)$).

RCM geometry. Following the reference paper’s shaft-parameterization idea, the candidate RCM point slides along the fixed-length tool segment:

$$p_{\text{RCM}}(q, \lambda) = p_{\text{base}}(q) + \lambda L \hat{z}(q), \quad \lambda \in [0.02, 0.995]. \quad (1)$$

λ is stored as the seventh state component, integrated like a joint angle. A key structural difference from the reference: the ICRA paper enforces $p_{\text{RCM}} \equiv p_{\text{trocar}}$ as a *hard* velocity constraint within the extended-task Jacobian; here “RCM at entry” (or “at target” in Task 2) is instead a *soft, gain-weighted point-regulation task* stacked alongside the others, which can be transiently outweighed by competing tasks (sections 8 and 9).

Task-space regulation and DLS solve. Every controlled quantity is a 3-D point task $y_i = f_i(q, \lambda)$, $\dot{y}_i = J_i \dot{x}$. Each active task’s error $e_i = y_{i,d} - y_i$ is magnitude-clipped and turned into a pure-proportional desired velocity $v_i = k_i \text{sat}_{\varepsilon_i}(e_i)$ – no feedforward term is ever added, even for the time-varying cone references of Tasks 2 and 4, which directly explains their non-zero steady-state tracking error (section 7). For all modes except Task 2’s strict-priority solve, active tasks are stacked ($J = [J_1; \dots; J_m]$, $v = [v_1; \dots; v_m]$) and solved by damped least squares [2]:

$$\dot{x}_{\text{base}} = J^\top \left(J J^\top + \mu^2 I \right)^{-1} v, \quad (2)$$

plus an approximate null-space secondary term $\dot{x} = \dot{x}_{\text{base}} + s_{\text{ns}}(w - J^\top (J J^\top + \mu_b^2 I)^{-1} (J w))$, where w combines a joint-limit-avoidance gradient, a λ -regulation term toward a mode-dependent nominal λ_d , and a skull-avoidance gradient. The skull term is only active when `useSkullAvoidance` is true *and* a skull exists; in the scene actually used, this flag is hard-coded `false`, so the term is inactive throughout the analysed run despite the skull sphere being present (section 9).

Priority. Tasks 1, 3, 4 use flat gain-weighted stacking (no exact hierarchy). Task 2 alone uses *strict* task priority [3]: the tip task is solved first, a tip null-space projector is built, and the entry-cone task is solved within it, with a final correction pass. Task 2 also implements an active-set guard on joint 4 (removes it from the stack if close to its limit); this guard never engaged in the analysed run.

Singularity handling. No SVD or condition number is computed. Instead, after each DLS solve the worst pre-saturation joint-speed ratio $\rho = \max_i |\dot{x}_i|/\dot{q}_{\max}$ ($\dot{q}_{\max} = 80^\circ \text{s}^{-1}$) is checked; if $\rho > 1$, the solve is repeated with boosted damping $\mu' = \mu\sqrt{\rho} \cdot 1.2$. Hard saturation follows ($|\dot{q}_i| \leq 80^\circ \text{s}^{-1}$, $|\dot{\lambda}| \leq 0.55 \text{s}^{-1}$), then explicit-Euler integration sub-stepped at up to 8ms, with joint/ λ clamping after every sub-step.

5 Implemented Surgical Tasks

Table 2: Point tasks stacked per mode (`BuildTasks()`). ✓ marks the strict-priority solve.

Task	Point tasks (order = priority for T2 only)	Strict priority
T1 – Entry-RCM, tip target	tip→target ($k = 3.5$); RCM→entry ($k = 17.0$)	–
T2 – Target-RCM, entry cone	tip→target ($k = 3.5 \times 1.65$); entry-side→cone ($k = 6.0 \times 0.70$)	✓
T3 – Safe insertion	phase-dependent, see below	–
T4 – Entry-RCM, tip cone	tip→cone ($k = 3.5 \times 0.85$); RCM→entry ($k = 17.0 \times 1.6$)	–

Task 2 (target-centred RCM, entry cone). The tip is the primary task, held at the deep target ($\lambda \rightarrow 0.995$). The entry-side cone reference is $p_{\text{cone},2}(t) = p_{\text{entry}} + r_2(\cos \phi \hat{b}_1 + \sin \phi \hat{b}_2)$, $r_2 = d_{\text{ET}} \tan \theta_c \approx 8.79 \text{ mm}$ ($d_{\text{ET}} = 111.7 \text{ mm}$, $\theta_c = 4.5^\circ$), $\phi(t) = \omega_2 t^+$, $\omega_2 = 8^\circ \text{s}^{-1}$ (deliberately lowered from an earlier 12°s^{-1} to keep tip-target error under threshold). An explicit `RCMPoint→target` task was removed from the stack (duplicated the tip task, caused jitter); `target_rcm_error_mm` is therefore a diagnostic, not an actively minimized quantity.

Task 3 (safe insertion). A finite-state machine with phases `ApproachEntry` (tip → pre-entry point 10 cm outside the body; no RCM task), `PierceEntry` (tip interpolates to the entry point; soft RCM pre-stabilization added past 80% progress; λ force-set to 0.995 on exit), `InsertToTarget` (RCM task active with boosted gain $\times 1.6$, tip gain reduced $\times 0.85$, favouring pivot stability over insertion speed), and `Done` (holds the final pose). A continuous secondary “insertion-axis guide point” task runs throughout all three active phases to keep the shaft collinear with the entry-target line.

Task 4 (entry-centred RCM, tip cone). The entry-RCM task stays active (gain $\times 1.6$) while the tip traces a cone of fixed radius $r_4 = 13.8 \text{ mm}$ around the target, $p_{\text{cone},4}(t) = p_{\text{target}} + r_4(\cos \phi \hat{b}_1 + \sin \phi \hat{b}_2)$, $\omega_4 = 7^\circ \text{s}^{-1}$ (slower than Task 2’s ω_2 , and null-space scale reduced $\times 0.35$) so the RCM task wins the flat-weighted conflict with the moving cone reference.

Transitions between Tasks 1–4 are operator-triggered keyboard events; only the internal Task-3 phase transitions are autonomous. Every mode switch has a shared 1.5 s cone warm-up delay before the cone phase angle begins to sweep.

6 Experimental Methodology

All quantitative results come exclusively from `RCM_logs/project4_multircm_20260714_193548_422.csv` (2696 samples, 157.258 s), one continuous session visiting modes 3→1→2→4 with no restarts. The `RCM_logs/` directory contains 21 other logs from the same session; none were used for any statistic in this report. Segmentation (table 3) was derived directly from the CSV’s `task/phase` columns.

Table 3: Pipeline segmentation from the selected CSV. n : sample count.

Task/Phase	Duration [s]	n
T3 – ApproachEntry	1.03	19
T3 – PierceEntry	6.21	117
T3 – InsertToTarget	9.96	180
T3 – Done	9.33	174
T1 – Entry-RCM, tip target	3.94	71
T2 – Target-RCM, entry cone	64.37	1097
T4 – Entry-RCM, tip cone	61.99	1038

Two thresholds come directly from the controller’s public fields (echoed in the CSV header): `entryRcmOkThresholdMm` = 2.0 mm and `tipTargetOkThresholdMm` = 3.0 mm, approximating clinical stereotactic accuracy. The skull threshold is 0 mm. A 50 mm cone-tracking reference line is used only as a loose sanity-check, not a pass/fail criterion. Following the analyzer’s convention, entry-RCM and tip-target statistics exclude the first 1 s after each switch (“settled” window); the full-phase (transient-inclusive) maximum is always reported alongside it.

7 Results

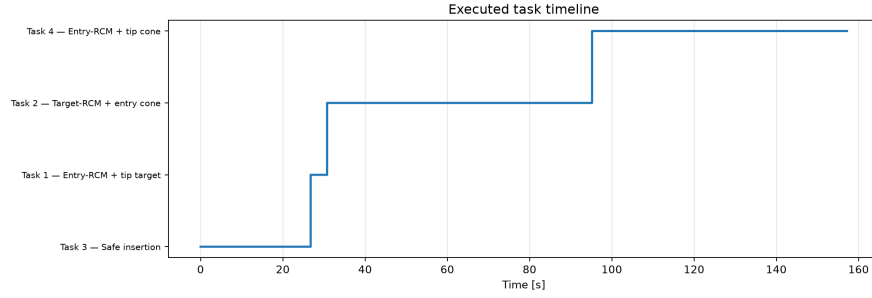


Figure 2: Executed task timeline, whole run ($t \in [0, 157.3]$ s). Steps mark manually triggered mode switches; no automatic supervisor chains the tasks.

7.1 Task 2 – Target-Centred RCM, Entry Cone

Over the full 64.37 s segment ($n = 1097$), the entry-cone tracking error had mean 4.96 mm, RMS 6.13 mm, median 4.17 mm, 95th percentile 13.17 mm, max 13.67 mm against a commanded radius $r_2 \approx 8.79$ mm – comparable in magnitude to the radius itself (fig. 3). The tip-target error (primary task) had mean only 0.006 mm, max 0.26 mm, confirming the tip stays essentially frozen at the target while the cone task absorbs the tracking lag. The diagnostic `target_rcm_error_mm` remained near-constant at 1.42 mm (std 0.003 mm), consistent with a static geometric offset rather than an actively minimized quantity.

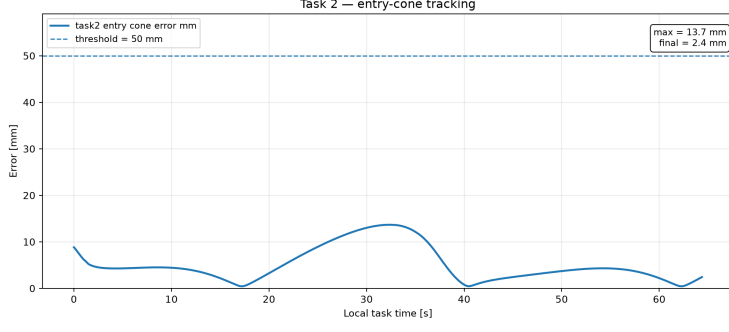
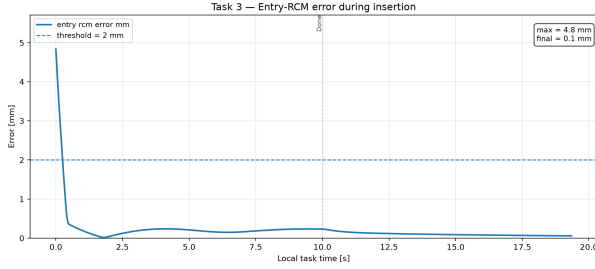


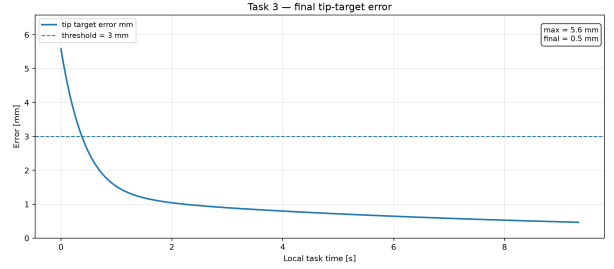
Figure 3: Task 2, entry-cone tracking error (mm) vs. local task time, full 64.37s segment. Dashed line: 50 mm sanity-check reference (not a threshold); commanded cone radius itself is ≈ 8.79 mm.

7.2 Task 3 – Safe Insertion

During **InsertToTarget** ($n = 180$ full / 160 settled), entry-RCM error reached a full-phase max of 4.84 mm (mode-switch transient) but, settled, had mean 0.179 mm, RMS 0.188 mm, max 0.237 mm – well inside the 2 mm threshold. During **Done** ($n = 174$ full / 155 settled), it settled further to mean 0.091 mm, max 0.151 mm. The final tip-to-target error (last logged sample) was 0.467 mm (settled-window mean 0.752 mm, max 1.498 mm), inside the 3 mm threshold. **skull_violation_mm** was identically zero across all 490 Task-3 samples; section 9 explains why this alone is not evidence of collision safety. Tool-axis angular error had median 1.08° over **InsertToTarget+Done**, dropping to 0.27° at the final sample.



(a) Entry-RCM error, **InsertToTarget+Done**.

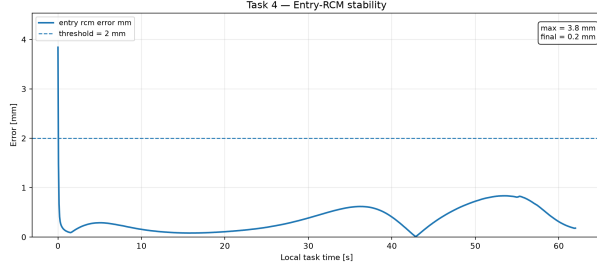


(b) Final tip-to-target error, **Done** phase.

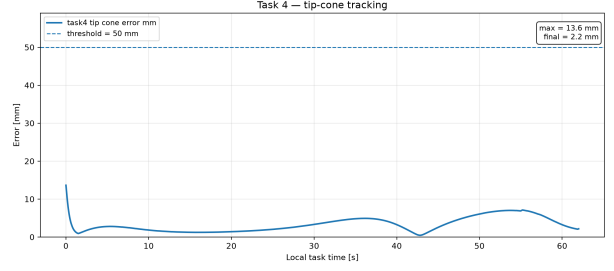
Figure 4: Task 3, safe insertion. (a) Entry-RCM error (mm), threshold 2 mm: settled max 0.237 mm. (b) Tip-to-target error (mm), threshold 3 mm: final value 0.467 mm.

7.3 Task 4 – Entry-Centred RCM, Tip Cone

Over the full 61.99s segment ($n = 1038$), settled entry-RCM error ($n = 1021$) had mean 0.353 mm, RMS 0.425 mm, max 0.834 mm (inside the 2 mm threshold), while the full-phase max was 3.84 mm (mode-switch transient). The tip-cone tracking error had settled mean 3.19 mm, RMS 3.67 mm, max 7.11 mm against the commanded $r_4 = 13.8$ mm radius – tighter, relative to the commanded radius, than Task 2’s entry-cone tracking, consistent with the reduced cone speed and gain rebalancing of section 5.



(a) Entry-RCM error.



(b) Tip-cone tracking error.

Figure 5: Task 4, full 61.99 s segment. (a) Entry-RCM error (mm): settled max 0.834 mm, transient max 3.84 mm. (b) Tip-cone tracking error (mm) against $r_4 = 13.8$ mm: settled mean 3.19 mm, max 7.11 mm.

7.4 Velocity and Runtime Diagnostics

Across the whole run, commanded joint-velocity norm $\|\dot{q}\|$ had mean 0.063 rad/s and max 1.25 rad/s, approaching but not reaching the $80^\circ \text{s}^{-1} \approx 1.396 \text{ rad/s}$ per-joint saturation limit. λ ranged from 0.601 to 0.995. Editor frame rate ranged 5.8 Hz to 213.0 Hz (mean 58.4 Hz) – an editor/host-load artifact, not a real-time guarantee. Jacobian-conditioning diagnostics (condition number, $\sigma_{\min}$) are implemented in the analyzer but not produced here, as the required columns are absent from the current CSV logger.

7.5 Summary Table

Table 4: Selected rows of the full metrics table (`report/final_metrics_table.csv`, 29 rows). “Settled” excludes the first 1 s after each switch; “–” = not pass/fail-gated.

Task / phase / metric	Unit	Mean	Max	Thr.	Result
T3 InsertToTarget (settled) entry-RCM error	mm	0.179	0.237	2.0	PASS
T3 Done (settled) entry-RCM error	mm	0.091	0.151	2.0	PASS
T3 Done (settled) tip-target error	mm	0.752	1.498	3.0	PASS
T3 all phases skull violation	mm	0.000	0.000	0.0	PASS*
T2 entry-cone tracking error (full)	mm	4.956	13.665	–	–
T2 tip-target error (settled)	mm	0.005	0.009	3.0	PASS
T4 entry-RCM error (settled)	mm	0.353	0.834	2.0	PASS
T4 tip-cone tracking error (settled)	mm	3.194	7.109	–	–
T1 entry-RCM error (brief, transient-dominated)	mm	0.076	0.088	–	–

*Skull avoidance is inactive in this run and the corridor is exempted from the check by construction; PASS reflects the logged value, not a validated collision-safety guarantee (section 9).

8 Comparison with the ICRA 2013 RCM Approach

Table 5: Aspect-by-aspect comparison between Aghakhani et al. (ICRA 2013) [1] and this project.

Aspect	ICRA 2013 paper	This project
Application	Endoscopic-camera visual servoing; obstacle-avoiding path following	Needle/electrode positioning for stereotactic neurosurgery
Robot platform	KUKA KR5 (Webots); DLR MIRO	Procedural ROSA-like 6R arm (Unity, no physics engine)
RCM definition	Slides along segment between two DH frame origins	Slides along the fixed-length rigid-tool segment (same idea)
RCM location	Fixed at entry/trocar	Entry-centred (T1/3/4) and target-centred (T2)
Task combination	RCM as hard velocity constraint in one extended Jacobian	RCM as a soft, gain-weighted task, flat (T1/3/4) or strict-priority (T2 only)
Jacobian	Analytic	Purely numerical, central finite differences
Singularity handling	Not addressed explicitly	Adaptive damping triggered by a joint-speed-saturation proxy
Joint limits	Not modelled	Hard DH clamp + soft null-space term + T2-only joint-4 guard
Safety modelling	RCM only (+ null-space obstacle avoidance in 9-DOF example)	Skull-avoidance term implemented but disabled; always-on diagnostic with corridor exemption
Validation	Simulation only, single/few runs	Simulation only, one continuous run across all four modes
Main limitation	No hardware validation; no joint-limit/singularity treatment	Kinematic-only; RCM is soft, so transiently violated at mode switches

The paper’s classical entry-port RCM corresponds directly to this project’s entry-centred mode (T1/3/4); the target-centred mode (T2) has no direct counterpart and is an untested extension of the same segment-parameterization idea with the roles of fixed/moving point swapped. Switching between RCM placements here is realized purely by changing which point tasks are stacked, exploiting a task-agnostic finite-difference Jacobian pipeline – the paper does not consider switchable RCM placements. Both works track moving references (conical/path-following) with pure proportional feedback and no explicit feedforward, so the steady-state cone-tracking lag reported in sections 7.1 and 7.3 is a direct, expected consequence of that shared choice, not an implementation defect. The paper’s Jacobians are analytic with undetailed damping/singularity handling; this project trades that rigor for a fully numerical pipeline plus an explicit adaptive-damping trigger, at a small constant per-step cost. Neither work presents hardware validation, and no cross-comparison of absolute accuracy is claimed given the differing platforms and scope.

8.1 Why These Choices Were Made

None of the deviations from the reference formulation are arbitrary: each follows from a concrete requirement of this project that the paper’s two worked examples did not have to satisfy.

Soft, gain-weighted RCM task instead of a hard velocity constraint.

The paper enforces one fixed RCM placement per experiment, so a single dedicated extended-Jacobian block is sufficient. This project instead needed *four* modes sharing one controller – entry-centred, target-centred, a phased insertion, and a cone motion – switched live by the operator. Treating the RCM condition as one more point task, with its own gain, lets `BuildTasks()` assemble any mode by simply choosing which points to stack and with what weight, without a different Jacobian-block structure per mode. The cost, an RCM that can be transiently outweighed by a competing task, was accepted because the settled-state results (section 7) show it converges to sub-millimetre accuracy in practice for the tuned gains used here.

Purely numerical (finite-difference) Jacobians instead of analytic ones.

This project has many more distinct task points than the paper’s examples (tip, entry-RCM, target-RCM, two different cone points, an insertion-axis guide point), each only active in some modes/phases. Deriving and maintaining a correct analytic Jacobian for every point, and keeping all of them consistent whenever the DH table or a task-point definition changed during development, was judged more error-prone and slower to iterate on than one generic central-difference routine that is automatically correct for any point defined through the same forward-kinematics chain. The trade is a small, constant extra cost (7 extra forward-kinematics evaluations per solve) against a much smaller surface for bugs.

Strict task priority only for Task 2, flat weighting elsewhere.

Task 2 is the one mode where the tip *must* stay essentially motionless at the deep target while a secondary motion happens at the entry side – exactly the situation exact priority is designed for, since it guarantees the lower-priority cone task cannot leak into the tip’s accuracy. Tasks 1, 3, and 4 do not have this same asymmetric criticality (their competing tasks are both required to converge, not one frozen and one moving), so the extra implementation cost of a strict-priority solve – a dedicated null-space projector, a correction pass, and its own singularity re-solve – was reserved for the one case that actually needed it, and gain tuning was used to manage the RCM/tip trade-off in the other three.

Heuristic adaptive damping instead of an SVD/condition-number-based singularity index.

Every DLS solve in this project already runs inside a per-frame loop that can be sub-stepped up to 8 times (section 4), and every sub-step already pays for a full finite-difference Jacobian rebuild. Adding a singular-value decomposition on top of that, potentially several times per rendered frame, was judged an unnecessary real-time cost when a much cheaper proxy – the worst pre-saturation joint-speed ratio – flags the same near-singular configurations (a joint being asked to move faster than it can) in time to boost damping before saturation clips the command.

A target-centred RCM mode (Task 2) not present in the paper.

The paper only demonstrates the entry-port RCM. Once the controller represents the RCM point as $p_{\text{base}}(q) + \lambda L \hat{z}(q)$, nothing in the geometry actually requires λ to be driven toward the entry side: driving it toward $\lambda \approx 1$ instead relocates the same machinery’s practical role to the target. This was added because it models a clinically plausible complementary scenario – the tip already at the target while a small, controlled motion is still needed at the skin/entry side – using the identical λ -based formalism, rather than a second, unrelated control law.

A three-phase insertion state machine instead of the four-phase one originally planned.

A separate `AlignAtEntry` phase was planned to align the shaft with the entry-target line before insertion, but was dropped in favour of a continuous secondary “insertion-axis guide point” task that runs throughout `ApproachEntry`, `PierceEntry`, and `InsertToTarget`. Gating alignment behind its own phase would have added another transition condition (and another transient at that boundary) for a behaviour that a continuously active secondary task already achieves smoothly.

Hard joint limits, null-space avoidance, and a Task-2-only joint-4 guard, none of which the paper models.

The paper’s visual-servoing and motion-planning examples do not report joint-limit handling. This project’s insertion task, by contrast, sweeps the arm between a pose 10 cm outside the body and a deep insertion pose, a much larger excursion where hitting a joint limit is a real risk; hard clamping plus a soft null-space avoidance term were added specifically to keep that excursion feasible, and the joint-4 active-set guard was added only for Task 2 because that is the one mode whose strict-priority solve was, during development, observed to occasionally drive joint 4 toward its limit.

Manually tuned, mode-specific gain multipliers instead of a single global gain set or a systematic optimization.

Several multipliers (e.g. Task 2’s cone-speed reduction from 12°s^{-1} to 8°s^{-1} , documented in the code’s own comments) were adjusted iteratively in direct response to a specific observed failure

mode – in that case, the tip-target error exceeding the 3 mm threshold at the faster cone speed. Given the project’s simulation-only, single-scenario scope, this iterative approach was judged sufficient to satisfy the clinically motivated thresholds for the analysed geometry, at the acknowledged cost of not generalizing automatically to a different entry/target configuration (section 9).

9 Discussion, Limitations, and Future Work

Precision and transients. Once settled, entry-RCM error stayed below 0.9 mm in every regulated task, an order of magnitude inside the 2 mm threshold, and the final insertion error (0.467 mm) is well inside 3 mm – for this geometry and gain set, the soft-task RCM formulation behaves close to a hard constraint, though this is an emergent, tuned-gain property, not a structural guarantee. Every mode switch produces a transient spike in the newly regulated error (4.84 mm at `PierceEntry`→`InsertToTarget`, 3.84 mm at `T2`→`T4`), decaying within 1 s: a single continuous run captures real transient behaviour that a steady-state-only summary would hide.

RCM-vs-cone trade-off. Tasks 2 and 4 both couple a fixed-point task with a moving cone reference; the code’s gain multipliers are direct evidence of hand-tuning this trade-off (T2: cone gain $\times 0.70$, tip gain $\times 1.65$; T4: tip-cone gain $\times 0.85$, RCM gain $\times 1.6$). The measured cone errors (T2 mean 4.96 mm/8.79 mm radius; T4 mean 3.19 mm/13.8 mm radius) show the trade-off is real: the pure-proportional law has no feedforward to compensate a continuously moving reference, and protecting the higher-priority task necessarily slows the lower-priority one further.

Single-run caveat. Every number in section 7 comes from one continuous, operator-paced run; sample counts are large (up to 1097 for T2) but temporally correlated, not independent trials. Results should be read as “this run satisfied these thresholds,” not a validated success rate across conditions.

Key limitations.

- Purely kinematic: no dynamics, actuator bandwidth, or force/torque control; velocities are integrated directly into positions.
- No tissue interaction/deformation; the skull is a rigid static sphere for a soft cost/violation check only, no physical trocar model.
- Skull avoidance is implemented but hard-coded `false` at scene-build time; the always-zero `skull_violation_mm` reading reflects avoidance being off and a corridor exemption that structurally excludes on-axis motion – *not* evidence of collision safety.
- No sensor noise, calibration error, or communication delay anywhere in the loop.
- All Jacobians are finite-difference, not analytic, even where closed forms exist.
- Gains are manually tuned per mode, not systematically optimized; integration is explicit Euler, sub-stepped but not adaptive.
- Single selected run, no repeated trials or statistical evaluation; DH table is plausible, not the real manufacturer’s; no hardware or clinical validation.

Future work. Hardware or higher-fidelity physics validation with real DH parameters; ROS/ROS 2 integration; torque-level dynamics-aware control; tissue-interaction/force feedback replacing the static skull sphere; an active always-on collision-avoidance term; formulating RCM and joint/velocity limits as hard constraints in a QP rather than soft tasks; a consistent hierarchical priority scheme across all four modes; principled/optimized gain tuning; robustness studies (noise, registration error, latency); repeated trials across postures and geometries with proper statistics; and a controlled comparison against a hard-constraint or QP-based virtual-fixture RCM controller.

10 Conclusion

This report reconstructed, from the C# source of a from-scratch Unity controller, a purely kinematic, penetration-coordinate RCM scheme in the spirit of Aghakhani et al. (ICRA 2013), implementing entry-centred and target-centred RCM modes, a three-phase safe-insertion sequence, and an entry-centred conical tip motion. Every quantitative claim was computed from one se-

lected experimental log and independently re-verified. Within that run, all settled entry-RCM and tip-target statistics satisfy the 2 mm/3 mm clinical thresholds, while the conical-motion tracking errors in Tasks 2 and 4 reveal a real, gain-tunable trade-off between pivot stability and moving-reference tracking under pure proportional control. The comparison with the reference paper shows a project that trades the paper’s analytic, hard-constraint RCM formulation for a more flexible, purely numerical, multi-mode, soft-task architecture – a reasonable engineering trade-off for a simulation-only, exploratory implementation, not a claim of improving on the reference method.

A Controller Parameters and Reproducibility

Table 6: Key controller gains and thresholds as coded (Inspector defaults).

Parameter	Value	Parameter	Value
Base tip / RCM / cone gain	3.5 / 17.0 / 6.0	T2 tip/cone/damp/ns mult.	$\times 1.65/0.70/1.0/0.6$
Base damping μ_0	0.055	T4 tip/RCM/damp/ns mult.	$\times 0.85/1.6/0.7/0.35$
Max joint speed / $\dot{\lambda}$	80° s^{-1} / 0.55 s^{-1}	Cone half-angle θ_c	4.5°
Singularity trigger / boost	1.0 / 1.2	Cone speed ω_2/ω_4	$8/7^\circ \text{ s}^{-1}$
Entry-RCM / tip-target thr.	2.0/3.0 mm	Task-4 cone radius r_4	13.8 mm

Reproducibility. Source log: `RCM_logs/project4_multircm_20260714_193548_422.csv` (fixed, not glob-selected). Pipeline: `.venv/bin/python3 report/scripts/generate_final_run_metrics.py` calls the unmodified `analyze_rcm_logs_clean.py` on this file, writing figures to `report/generated_figures/` and `final_metrics_table.csv`; then `cd report && tectonic report.tex` compiles this document. Full details, including the ten-value independent numerical-verification pass, are recorded in `report/REPORT_GENERATION_NOTES.md`.

References

- [1] N. Aghakhani, M. Geravand, N. Shahriari, M. Vendittelli, G. Oriolo, “Task Control with Remote Center of Motion Constraint for Minimally Invasive Robotic Surgery,” *2013 IEEE International Conference on Robotics and Automation (ICRA)*, Karlsruhe, Germany, May 6–10, 2013, pp. 5807–5812.
- [2] Y. Nakamura, H. Hanafusa, “Inverse Kinematic Solutions with Singularity Robustness for Robot Manipulator Control,” *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, 1986.
- [3] B. Siciliano, J.-J. E. Slotine, “A General Framework for Managing Multiple Tasks in Highly Redundant Robotic Systems,” in *Proc. 5th Int. Conf. on Advanced Robotics (ICAR)*, 1991, pp. 1211–1216.